

EVICTED ADDRESS FILTER: A Unified Mechanism to Address Both Cache Pollution and Thrashing

Objective: Efficient cache utilization is critical in modern multi-core processors, especially under high memory pressure. The Evicted Address Filter (EAF) aims to mitigate two fundamental cache management issues. **Cache Pollution** and **Cache Thrashing**, by predicting the reuse behavior of missed cache blocks. EAF refines cache insertion policies, reducing both pollution and thrashing while maintaining low hardware overhead. The high hardware overhead of tracking all missed blocks is mitigated by leveraging the observation that blocks evicted a long time ago are likely low-reuse. This allows EAF to track only a small subset of evicted PCs.

Hardware Implementation: EAF is implemented as a separate hardware structure outside the cache, making it easy to integrate with existing systems. Unlike full-history tracking, EAF maintains a small set of recent evictions. EAF operates externally to the cache and affects only cache misses. Since EAF does not modify cache hit behavior, it can integrate seamlessly with existing replacement policies. Once an address is inserted into EAF, it remains until the entire filter is cleared. Instead of FIFO-based single-entry eviction, EAF is cleared completely when full, ensuring only high-reuse addresses persist.

Implementation in ChampSim:

EAF alters how blocks are inserted into the **Last-Level Cache (LLC)** based on eviction history.

1. **Modification of Cache Insertion and Replacement Policies** (replacement.cc)
 - Track evicted addresses.
 - Look up evicted addresses in the Bloom Filter.
 - Determine insertion priority accordingly.
2. **Implementation of EAF and Bloom filter:**
 - **Eviction Tracking:** Modify cache.cc to store addresses of evicted blocks.
 - **Bloom Filter Integration:** Implement bit vectors and hash functions (H3 class XoR functions) to efficiently store eviction history and improve look up latency.
 - **Handling Thrashing:** Implement periodic clearing of the Bloom Filter to ensure only high-reuse blocks persist using saturating counters.

Testing and Validation

- **Tracking Behavior:** Implement debugging metrics to analyze the movement of blocks in and out of EAF.
- **Benchmark Selection:** SPEC CPU2017, PARSEC, CloudSuite, MLPerf and TPCx-AI to better reflect modern workloads like AI, cloud computing, and big data processing.
- **Performance Metrics:** Evaluate improvements in IPC, cache hit rate, and memory latency on general-purpose, server, and database workloads.
- **Comparison with Prior Work:** Measure EAF performance against existing cache management policies such as **SHIP, DRRIP, and TA-DIP**.
- **Bloom Filter False positive rate:** Optimize Bloom Filter parameters (bit-array size, hash functions) to control the false positive rate.

- **Sensitivity Analysis:** Tune EAF parameters to balance storage overhead and prediction accuracy.
- **Integration Analysis:** Test EAF alongside advanced replacement policies (e.g., RRIP) to measure its orthogonality.

Alternative EAF Design Choices to be tested:

- **Segmented-EAF:** Instead of full-clear, segment the Bloom Filter and clear only a fraction when full based on FIFO group organization of the segments.
- **Decoupled-Clear-EAF:** Increase EAF size beyond the cache block count to increase visibility of evicted PCs and reduce Cache Pollution. Analyze cost-performance tradeoffs
- **Dueling-EAF (D-EAF):** Since a block's first access is a miss, it is initially predicted as low-reuse. This can degrade performance for LRU-friendly workloads. **D-EAF** assigns two groups of cache sets to different policies and adapts dynamically using counters.

References:

[1] Vivek Seshadri, Onur Mutlu, Michael A Kozuch, Todd C Mowry. “The evicted-address filter: a unified mechanism to address both cache pollution and thrashing”. PACT '12: Proceedings of the 21st international conference on Parallel architectures and compilation techniques; 355 – 366. [Unified Approach to address Cache Thrashing and Pollution](#)

[2] Nimrod Megiddo, Dharmendra S. Modha. “ARC: A Self-Tuning, Low Overhead Replacement Cache”. FAST '03: Proceedings of the 2nd USENIX Conference on File and Storage Technologies; 115 – 130. [ARC- Low Hardware Overhead Cache Replacement](#)

[3] Mazen Kharbutli, Yan Solihin. “Counter-Based Cache Replacement Algorithms”. ICCD '05: Proceedings of the 2005 International Conference on Computer Design; 61 – 68. [Counter-Based Cache Replacement Algorithms](#)

V-way cache for providing demand based associativity

The paper 'The V-Way Cache : Demand-Based Associativity via Global Replacement' aims at reducing conflict misses and providing best victim elimination. This requires maintaining more number of tag stores as compared to a corresponding set-associative cache of the same size. These extra tag stores are then dynamically mapped to data stores from different sets that are identified as less reused by the replacement policy. Thus the associativity of high demand sets increases at the expense of dormant low demand sets.

To identify low reuse blocks, a global replacement policy tracks reuse frequency of each block. Due to lower reuse locality of L2 as compared to L1, 75% of the blocks evicted from L2 have a reuse frequency of at most 1. The remaining highly reused blocks will benefit greatly from the increased associativity by mapping to these low-reuse blocks.

Implementing the V-way cache in Champsim requires adding (1)Extra tags;The exact number is determined by the TDR(Tag stores to Data stores ratio) which depends on how much maximum associativity is required, (2) Forward pointers (FPTR) for each tag-store entry, and (3)Reverse pointers(RPTR)+Reuse Counters for each data-store entry. The modifications are detailed as follows:

Modifying the tag store structure:

- Increase the number of tag-store entries relative to the number of data lines per-set.
- Modify the tag entries to hold the forward pointer (FPTR) that point to the data-blocks.
- Addition of FPTR selection logic in addition to tag comparison.

Global Reuse Replacement Policy

- Modify the data-store to hold reverse pointers (RPTR) that point to the tag store for every cache block.
- Identify the tag-victim from the target set of the tag store on a cache-miss.
- Implement logic to get the next data victim from the reuse counter table, if the tag-store contains any invalid entry.
- If no invalid tag store is found, resort to an updated LRU policy that finds tag victims using LRU, but uses FPTR based replacement.
- Update the RPTR of the data victim to point to the newly validated tag-store entry.

Constructing the Reuse Counter Table(RCT)

- Set up a 2-bit saturating reuse counter as the number of L2 accesses to a cache line.
- Implement pointer based strided traversal of RCT & decrement each non-zero counter until data-victim is found.

Testing and Validation

- Determine the ratio of the global replacement policy invocations over the number of cache misses for different workloads.
- Inspect the achieved power reduction due to serialization in tag & data access.
- Run simulations on ChampSim across a range of benchmarks, especially those having wide disparity in set demand.
- Analyze the impact on overall performance, instruction per cycle (IPC), and impact on cache-hit latency.

Design Choices and Optimizations

- Vary the Tag to Data Store ratio, to analyse the impact of providing maximum higher associativity.
- Use other replacement policies like PLRU, NMRU instead of LRU when no invalid tags are left
- Invalid tag-store entries can be used to maintain inclusion information without the need for duplicating cache lines in the data-store.

References:

- [1] D. Sanchez and C. Kozyrakis, "The ZCache: Decoupling Ways and Associativity," *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*, Atlanta, GA, USA, 2010, pp. 187-198. [The ZCache: Decoupling Ways and Associativity](#)
- [2] M. K. Qureshi, D. Thompson and Y. N. Patt, "The V-Way cache: demand-based associativity via global replacement," *32nd International Symposium on Computer Architecture (ISCA'05)*, Madison, WI, USA, 2005, pp. 544-555. [The V-Way cache: demand-based associativity via global replacement](#)
- [3] A. Jain and C. Lin, "Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement," *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, Seoul, Korea (South), 2016, pp. 78-89. [Leveraging Belady's Algorithm for Improved Cache Replacement](#)